B.Sc. Computer Science · Year 1, Semester 1

SCS1303

Introduction to Software Engineering

Section 1 · Software & Software Engineering

Prof. K. P. Hewagamage

University of Colombo School of Computing

Course Administration

How this course is organised

1 **Attendance**

Both theory and practical sessions are required

2 **UGVLE Course Page**

All announcements, slides, and tasks posted here

3 **Profile Picture**

Add to UGVLE so the class can get to know each other

4 **Asking Questions**

Direct (face-to-face) or via slido.com during lectures

5 **Tutorial Activities**

Weekly tasks — active participation expected

6 **Assessment**

Assignments 40% · Final exam 60%

Outline of Syllabus

1. Introduction
2. Software Processes
3. Requirement Engineering
4. Agile Software Development
5. System modeling
6. Architectural design
7. Design and implementation
8. Software Testing
9. Software evolution

Main References

1. 

Software Engineering by Ian Sommerville, 10th edition, Addison-Wesley, 2015.

2. 

Software Engineering: A practitioner's approach by Roger S. Pressman,
   - 9th edition, McGraw-Hill International edition, 2020.

Section 1 — Intended Learning Outcomes

By the end of this section, students should be able to:

LO1	Define software and software engineering using standard recognised definitions
LO2	Classify types of software and their application domains with real-world examples
LO3	Explain the characteristics that make software development uniquely challenging
LO4	Describe the software crisis and how it gave rise to software engineering as a discipline
LO5	Outline the key SDLC stages and the main roles within a software project team
LO6	Identify the professional and ethical responsibilities of software engineers
LO7	Describe how modern software is delivered — web, cloud, mobile, and AI-assisted environments

Core ILOs Current-world extension

1.1

The World Runs on Software

Starting with what you already know

- The economies of ALL developed nations are dependent on software.
- More and more systems are software controlled

What Software Did You Use Today?

Before arriving here this morning...

WhatsApp / Telegram

Communication

Google Maps / Waze

Navigation

Instagram / TikTok

Social Media

Mobile Banking App

Financial

Alarm / Calendar

Productivity

YouTube / Spotify

Media

You are already a software user. This course teaches you how to build, manage, and engineer it — professionally and responsibly.

1.2

What is Software?

Defining the subject of our discipline

Defining Software

IEEE Definition

"Computer programs, procedures, and associated documentation and data pertaining to the operation of a computer system."

Software is more than code — it has three essential components:

01 Instructions

Programs that execute to deliver desired features and performance

machine-readable instructions to the computer's processor to perform specific operations.

02 Data Structures

Organised data the programs manipulate and store

a collection of intercommunicating components, programs, configuration files

03 Documentation

Descriptions of operation, use, and design of the system

system documentation and user documentation

Software: Generic vs Bespoke

Two fundamental models of how software reaches users

Generic (Product)

- Developed for a broad market of customers
- One product sold to many — developer controls the spec
- Functionality specified by market research
- Microsoft Windows, WhatsApp, Photoshop, Google Chrome

vs

Bespoke (Custom)

- Built for a single customer to their specification
- Customer defines what it must do — higher cost per unit
- Fits the client precisely but is not resaleable
- Hospital management system, payroll system, airline reservations

Questions about software

- Why does it take so long to get software finished?
- Why are development costs so high?
- Why can't we find all errors before we give the software to our customers?
- Why do we spend so much time and effort maintaining existing programs?
- Why do we continue to have difficulty in measuring progress as software is being developed and maintained?

1.3

Types & Domains of Software

Understanding the software landscape

The Two Fundamental Types of Software

System Software

Operates and controls computer hardware; provides a platform for applications.

- Operating Systems — Windows, Linux, Android
- Device Drivers
- Compilers & Interpreters
- Database Management Systems
- Virtualisation Tools — Docker, VMware

Application Software

Performs specific tasks for end users, running on top of system software.

- Word processors and spreadsheets
- Business and accounting packages
- Web browsers
- Mobile applications
- Games and entertainment

Software Application Domains

Every domain has unique challenges — and uses the same core engineering principles

Embedded

Pacemakers, car engine control units, ATMs

Web & Mobile

Banking apps, e-commerce, social media

AI & Machine Learning

ChatGPT, fraud detection, recommendation systems

Engineering / Scientific

Climate models, simulations, data analysis

Business

ERP, CRM, payroll, inventory management

System Software

Operating systems, compilers, DBMS

Product-line

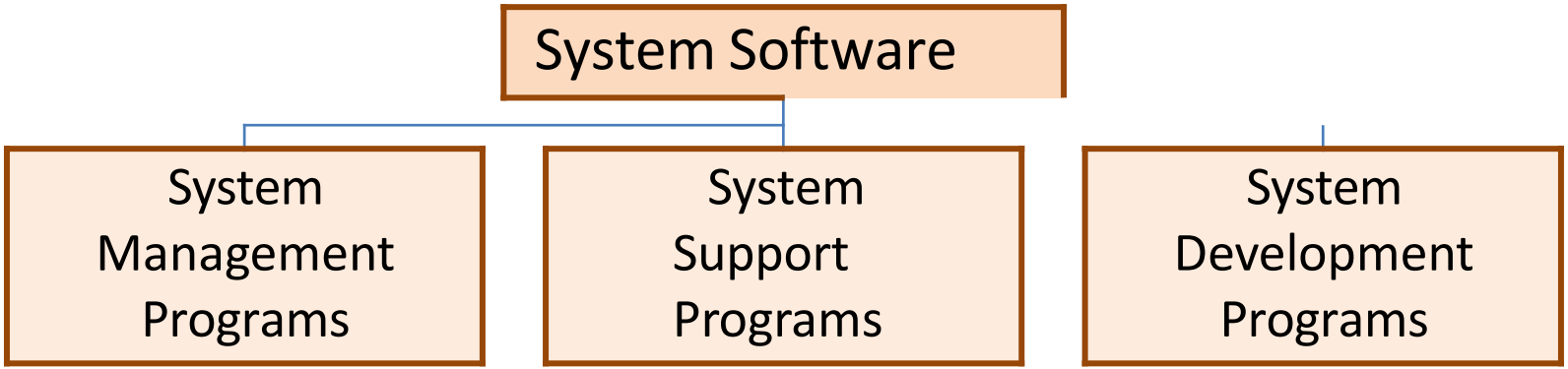
A family of related banking software products

Legacy

Old COBOL mainframe systems still in use today

Each domain uses the same engineering principles —
but demands different methods, tools, and quality priorities.

System Software

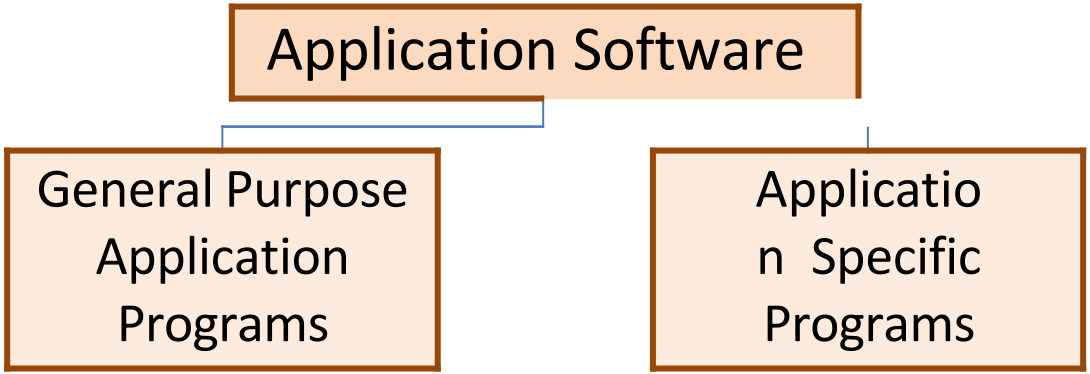


Common Software Types

- **System Software:**
  - System software is a collection of programs is written to service the other programs.

Eg: Operating system component, drivers, telecommunication process

Application Software



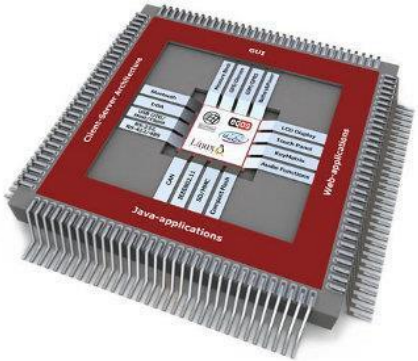
- Word Processing
- Electronic Spread Sheets
- Database Managers
- Graphics Software
- Integrated Packages

- Accounting, General Legers etc.
- Marketing-Sales Analysis etc.
- Manufacturing-Production Control etc.
- Finance-Capital Budgeting etc.

Common Software Types

- **Business software:**

  management information system software that access one or more large database containing business information



- **Embedded software:**

  Embedded software resides in read-only memory and is used to control product and system for the customer and industrial markets

Common Software Types

- **Web-based software:**

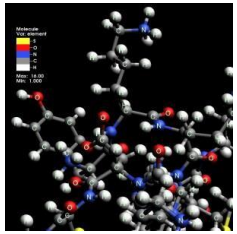
  The network becomes a massive computer providing an almost unlimited software resources that can be accessed by anyone with a modem.

- **Artificial intelligence software:**

  AI software makes use of non-numerical algorithms to solve the complex problems that are not amenable to computing or straightforward analysis.

Common Software Types

- **Scientific or Engineering Software :**
- Used for modeling, simulation, or analysis in fields like physics, chemistry, and engineering.



- **Personal computer software:**

  Personal computer software market has burgeoned over the past two decades.

  Word processing, spreadsheets, computer graphic, multimedia management

1.4

Why Software is Different

The unique challenges of a brain product

What Makes Software Uniquely Hard to Build?

Challenge	Why It Matters
Difficult to specify completely	Customers often do not know what they want until they see it
Requirements change constantly	Like building a house where the design keeps shifting mid-construction
Software is intangible	You cannot see or touch a bug the way you can see a crack in a bridge
Exhaustive testing is impossible	Every combination of inputs, conditions, and paths cannot be tested manually
Small changes have large effects	A change to a few lines of code can change the entire system's behaviour

Where Does the Money Go?

The economics of software over its lifetime

60–80%

of total software lifecycle cost goes to maintenance

Source: IEEE Std 1219 · Standish Group

Development **20–40%**

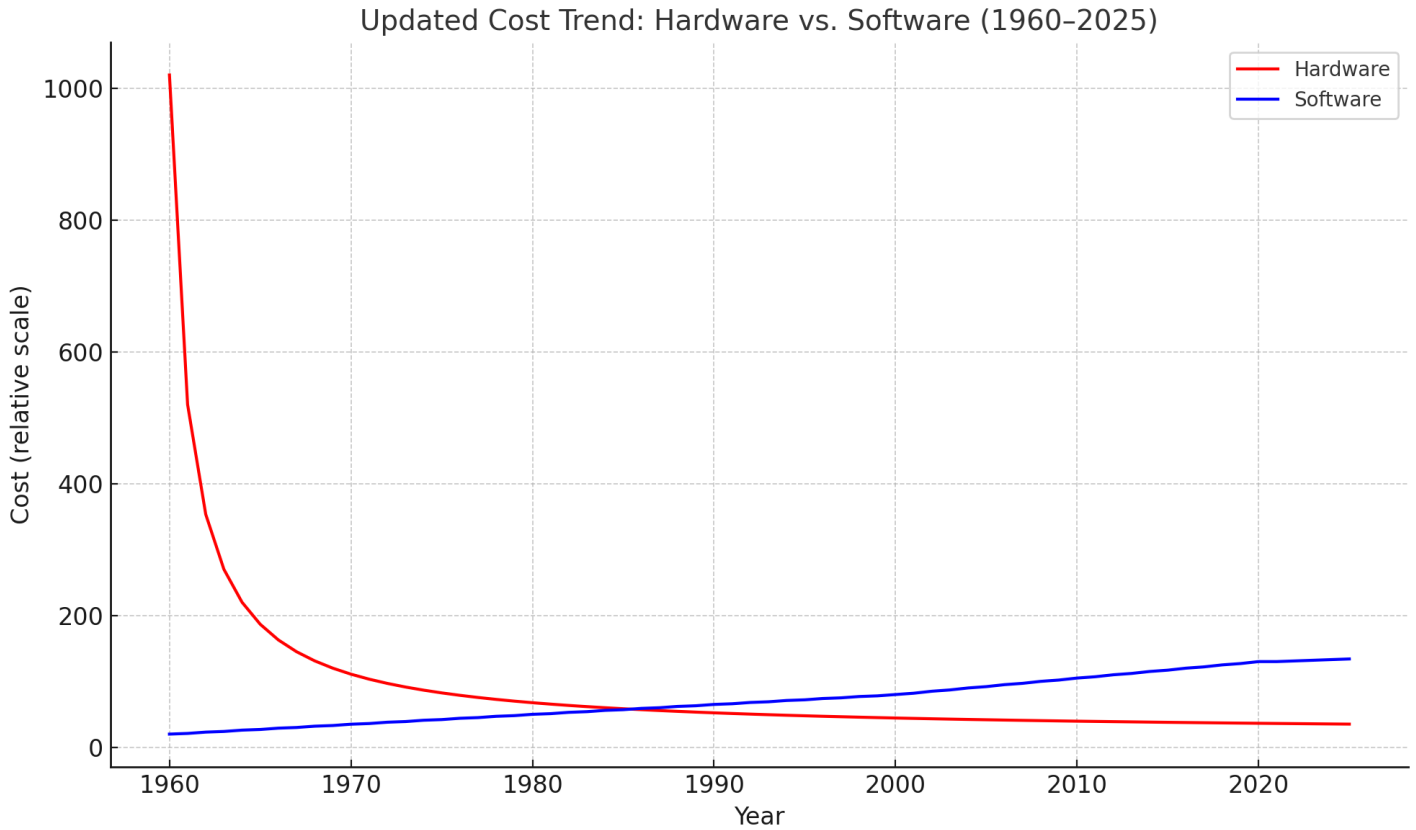
Requirements → Design → Code → Test → Deploy

Maintenance **60–80%**

Bug fixes, feature updates, compatibility, security patches over the product's life

"Writing code is only the beginning. Engineering software well determines what it costs for the next 10–20 years."

Cost of Hardware vs. Software



What is the trend after 2020?

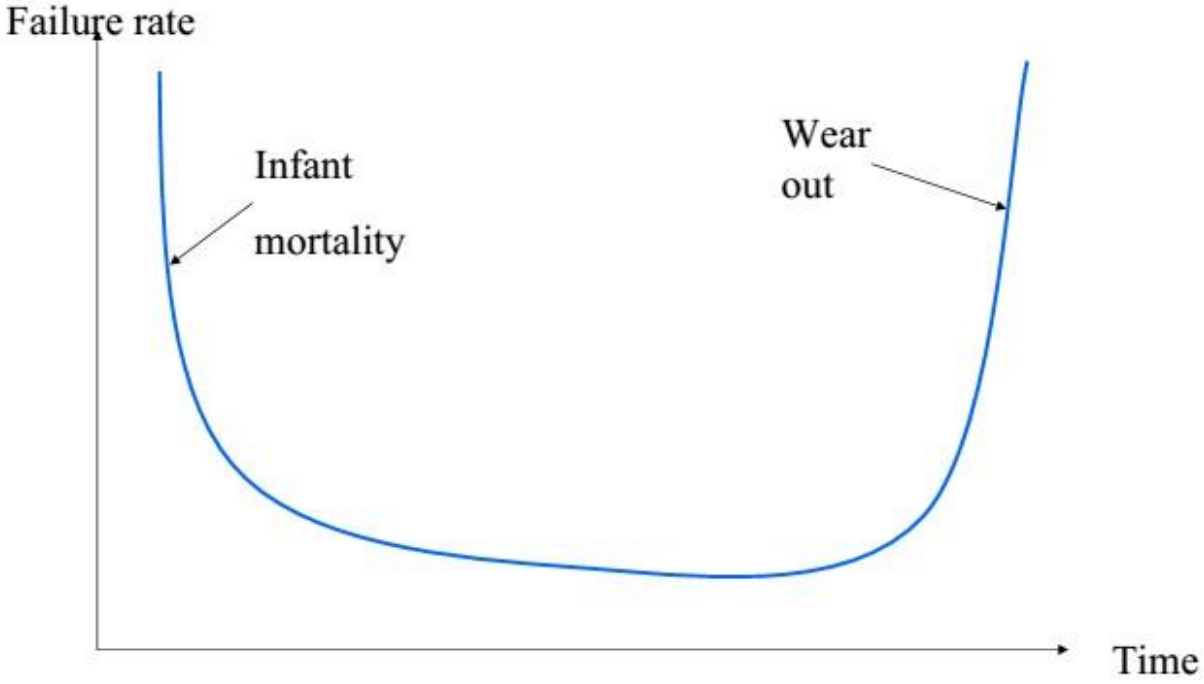
Software Costs : Why Software is More Expensive

- Software now dominates total system cost
  - For most modern systems, especially PCs and enterprise setups, the software licenses, services, and support outweigh the hardware costs.
- Maintenance > Development
  - Maintaining software (fixes, updates, upgrades, compatibility) can cost **2–5 times** more than the original development — especially in systems with long life cycles.
- Software Engineering is Cost Engineering
  - A major goal of software engineering is to make software that's reliable, maintainable, and scalable — without ballooning the cost.

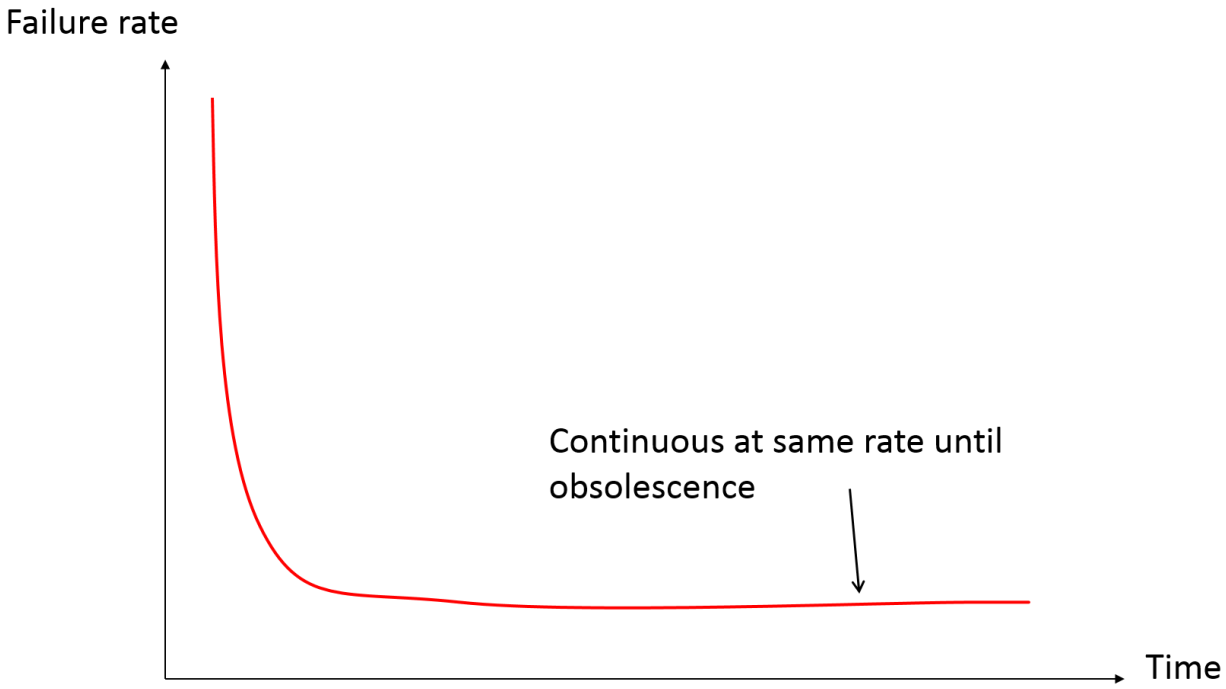
Failure Curves for Software vs. Hardware

- Software is engineered, not manufactured — no physical degradation
- Software doesn’t “wear out,” but it can still fail, especially when changed
- Hardware physically ages, while software logically degrades

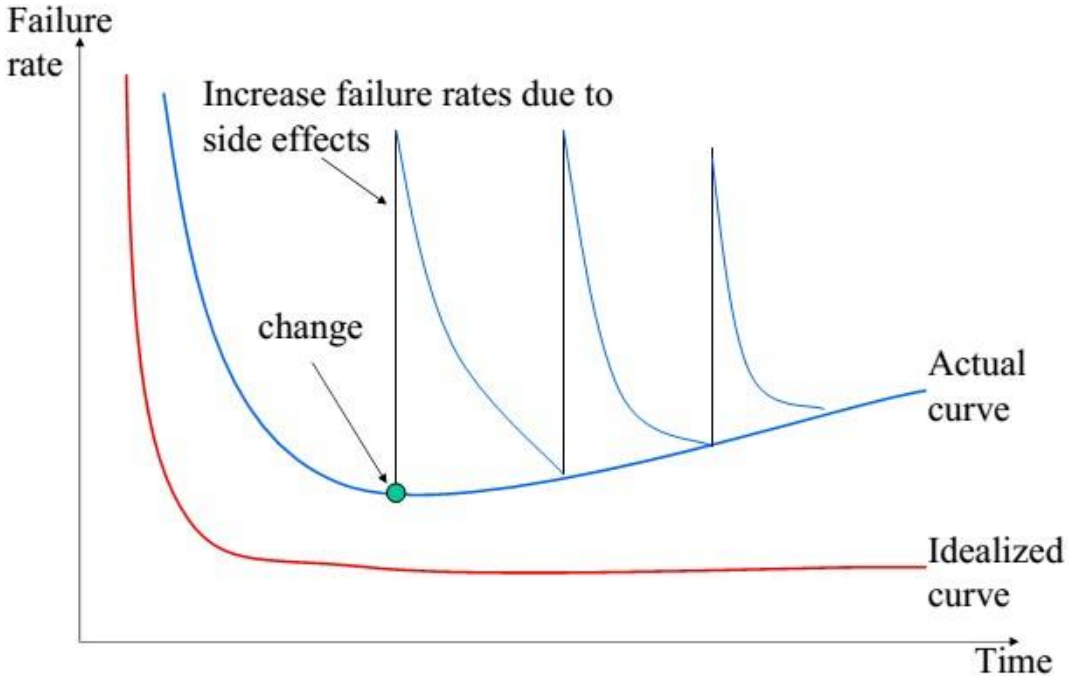
Failure “Bathtub” curve for hardware



Failure curve for software



Failure curve for software: in reality



How Software Fails vs How Hardware Fails

Hardware — Bathtub Curve

Hardware failure follows a predictable pattern over time:

- High early failure rate (infant / manufacturing defects)
- Low during useful life (steady, predictable operation)
- Rising again at end of life (physical wear and ageing)

Because hardware degrades physically, this pattern is well understood and engineers can design around it.

Software — Spike Pattern

Software does not wear out physically — but it still fails:

- Failure rate spikes each time the software is changed
- Every new feature or bug fix risks introducing new defects
- Complexity grows over time, making changes riskier

Software degrades logically, not physically. Good engineering practice — reviews, testing, clean design — controls this degradation.

1.5

The Software Crisis & Birth of SE

Why we needed a discipline, and what emerged

Why Is Building Software So Hard?

1. Why does it take so long to finish software?
2. Why are development costs so high?
3. Why can we not find all errors before delivery?
4. Why do we spend so much effort maintaining existing programs?
5. Why is it so hard to measure progress while software is being developed?

These questions — still relevant today — drove the search for a disciplined engineering approach to software.

The Software Crisis

Formally raised at the 1968 NATO Software Engineering Conference, Garmisch

As hardware became cheaper and faster, software could not keep pace. Developers lacked structured methods to manage growing complexity.

Over budget

55% of projects cost more than originally estimated

Late delivery

Average project overshot schedule by 50% or more

Wrong product

Software often failed to meet stated user requirements

Unmaintainable

Code was difficult to understand, modify, or scale

Never delivered

~50% cancellation rate for large software projects

No discipline

No formal engineering methods — just ad hoc coding

Real-World Software Failures

These are not historical curiosities — project failures still happen today

UK Home Office — National Probation Service System

Ran 70% over its original budget, ending at £118m. Hampered by a poorly specified requirements and a restrictive contract.

Poor requirements + rigid contract = cost disaster

UK Air Traffic Control — Swanwick Centre

Six years late and £180m over budget. Declared near-obsolete before it went operational due to rapidly changing technology.

Scope creep + complexity = schedule failure

Boeing 737 MAX — MCAS Software

Control system design flaws contributed to two fatal crashes in 2018–2019, grounding the fleet globally for over a year.

Safety-critical software demands rigorous engineering

The Classic Communication Problem

Discussion: Where does software go wrong?

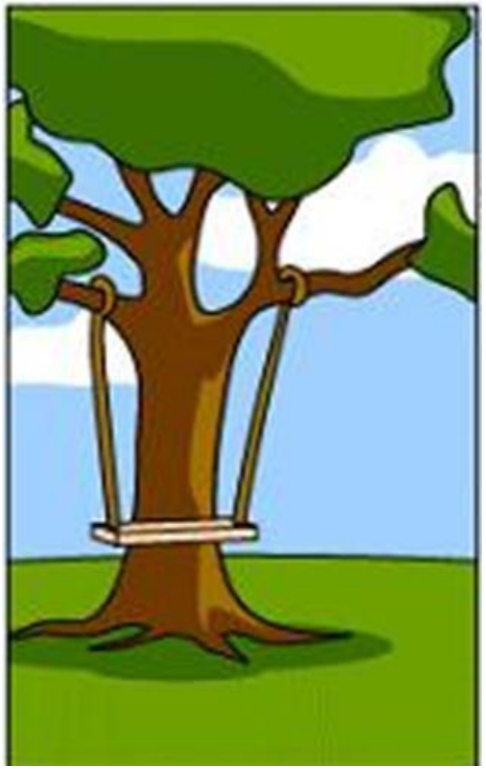
Consider what happens when different people involved in the same project interpret requirements differently.

1. How the customer described it
2. How the analyst understood it
3. How the architect designed it
4. How the developer coded it
5. What was actually deployed
6. **What the customer really wanted**

This classic illustration (Sommerville Ch 1) captures why communication, requirements engineering, and process discipline matter.

How the Customer Described it

How the Business Analyst Understood it

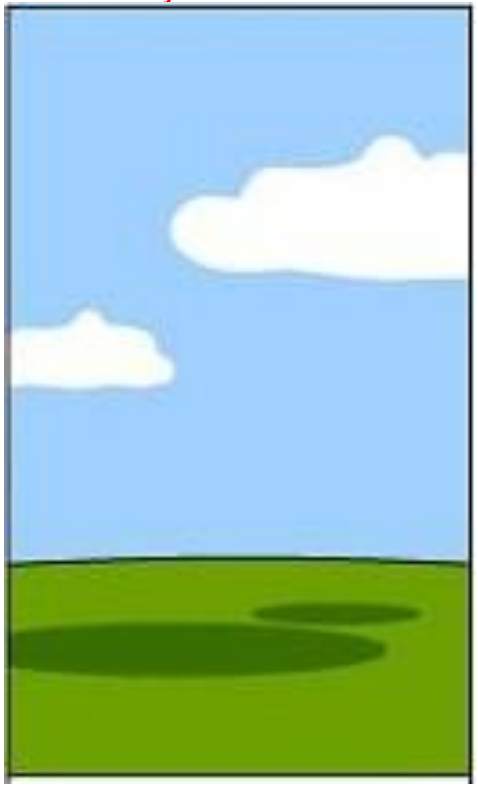


How the Architect Designed it

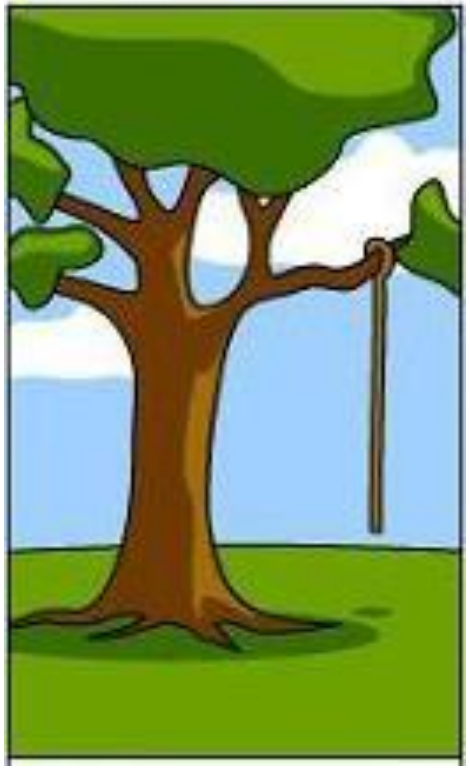
How the Developers coded it

How the Marketing team described it to the customer

How the Project was Documented

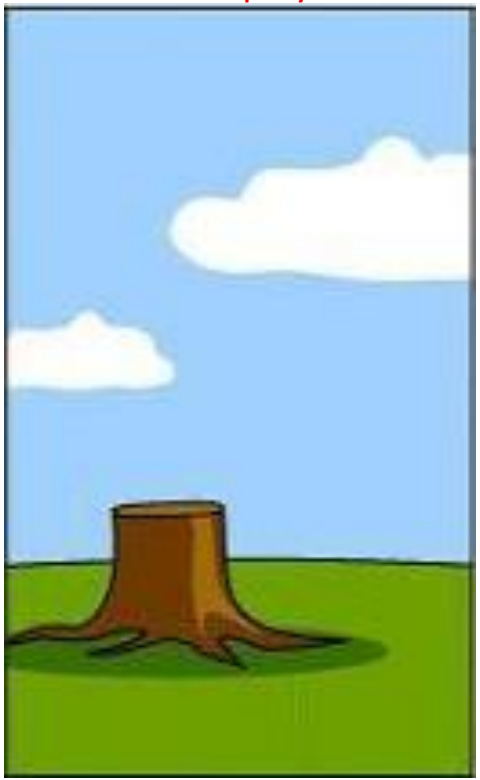


How it was Deployed at Customer Site



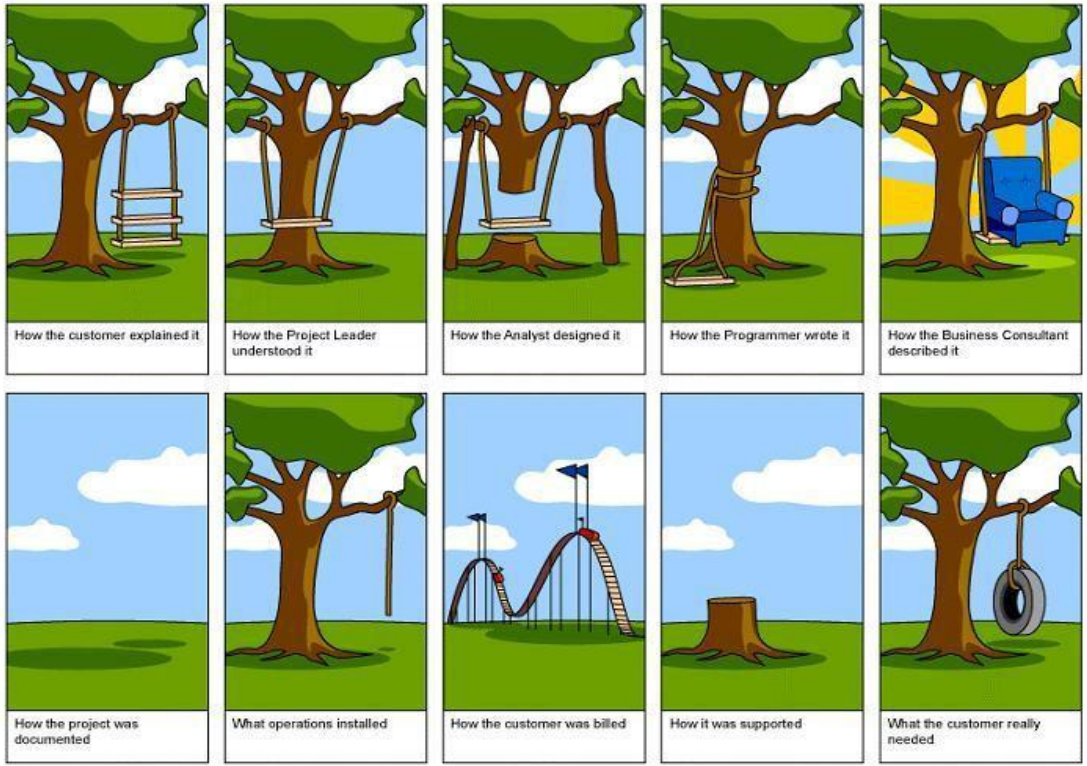
How the Customer was Billed

How it was Supported after Deployment



What Customers Really wanted!

How the Swing Project happened!!!



Software Engineering — The Solution to the Software Crisis

Area	What It Contributes
🎯 **Clear Requirements**	Understanding user needs precisely through formal/semi-formal methods
🧪 **Prototyping**	Demonstrating early system versions to reduce risks and gain feedback
🛠️ **Engineering Process**	Structured models (e.g., Waterfall, Agile) for planning and control
🔍 **Quality Focus**	Emphasis on readable, error-free code that meets expectations
💻 **Tool Support (CASE tools)**	Automated support for design, testing, documentation, and maintenance

What is Software Engineering?

IEEE 610.12 — Standard Definition

"The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software."

Systematic

Planned and structured — not guesswork or improvisation

Disciplined

Follows defined principles, standards, and good practice

Quantifiable

Outcomes can be measured, not just felt or assumed

Software engineering brings to software what civil or mechanical engineering brings to physical structures — structured process, sound design, rigorous testing, and professional accountability. It is concerned with theories, methods, and tools for professional software development.

Software Engineer · Developer · Computer Scientist

Related roles — but different focus and scope

Computer Scientist

Primary focus

Theory & Fundamentals

Key activities

- Algorithms and data structures
- Mathematical foundations of computing
- Research and computing theory

Software Developer

Primary focus

Coding & Building Features

Key activities

- Writes code and builds features
- Works at project or task level
- Implements what has been specified

Software Engineer

Primary focus

Systems & Engineering

Key activities

- Applies engineering principles
- System-wide and architectural thinking
- Plans, models, and optimises quality

This course focuses on software engineering —
the discipline that makes large, complex software reliable, maintainable, and deliverable.

Software Engineer vs Developer

Software Engineer vs Developer – What's the Difference?

Role	Focus	Scope	Thinking Style
Software Developer	Focuses on coding and building features	Project/task level	**Tactical** – implement what's given
Software Engineer	Applies engineering principles to system design	System-wide / architectural	**Strategic** – plan, model, optimize

Full Stack Developer?

a **Full Stack Developer** — someone who codes both frontend (UI/UX) and backend (servers, APIs).

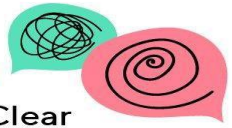
That's a **specialized developer** role — not necessarily an engineer, but highly skilled.

Top Skills for Software Engineers

1. Problem Solving

2. Attention to Detail

3. Clear Communication



4. Continual Learning

5. Teamwork

6. Empathize with End Users

UCSC

1. 🧠 **Problem Solving**
“At the core of engineering is problem solving — not just writing code, but finding the *right* solution for the *right* problem.”

2. 🔬 **Attention to Detail**
“One semicolon in the wrong place can crash a program. Precision is power.”

3. 🗣️ **Clear Communication**
“You’ll often explain technical ideas to non-technical people — clarity builds trust.”

4. 📚 **Continual Learning**
“Tech changes fast. You don’t stop learning after graduation. Every tool or language you know today might be outdated tomorrow.”

5. 🤝 **Teamwork**
“Modern software is built in teams — collaboration tools, code reviews, pair programming are all part of it.”

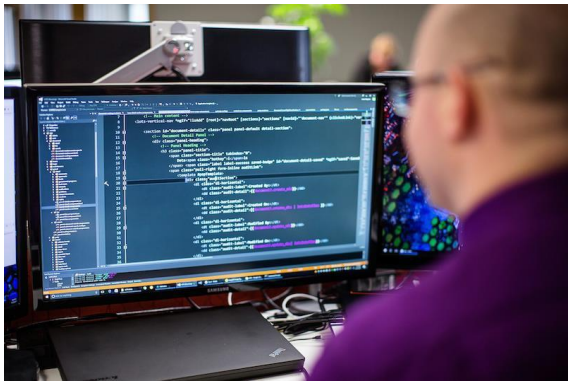
6. ❤️ **Empathize with End Users**
“User-centered thinking helps you build systems people actually want to use.”

1.6

SDLC, Project Teams & Quality

The structure behind software development

Software Projects vs. Other Projects



Software Projects vs. Other Projects

- It is difficult to test software exhaustively.
- Not like other engineering disciplines that have narrow scope, software can be developed in diverse contexts ranging from embedded systems to giant manufacturing systems.
- The replication of software is trivial and as a consequence software is elastic and gradually becomes more complex as changes are made over time.
- A small change to a few lines of code could do a big change in the overall behavior of the system.
- Finally, as a discipline, software development is so young that measurable, effective techniques are not yet available, and those that are available are not well-calibrated.

Why Software Projects Fail

- Increasing System Complexity

Modern software must:

- Support **larger and more complex systems**
- Be delivered **faster**
- Enable **new capabilities** once thought impossible

Bigger dreams bring bigger risks.

- Lack of Software Engineering Discipline

Many teams:

- Write code without **structured engineering principles**
- Skip formal **design, documentation, and testing**
- Focus on speed over **quality and maintainability**

Result: Projects may be fast and cheap but are often **unreliable and hard to scale**.

What is Software Development?

- Software development is the process of designing, programming, testing, and maintaining software products. It involves more than just writing code — it includes planning, documentation, and ongoing improvement.

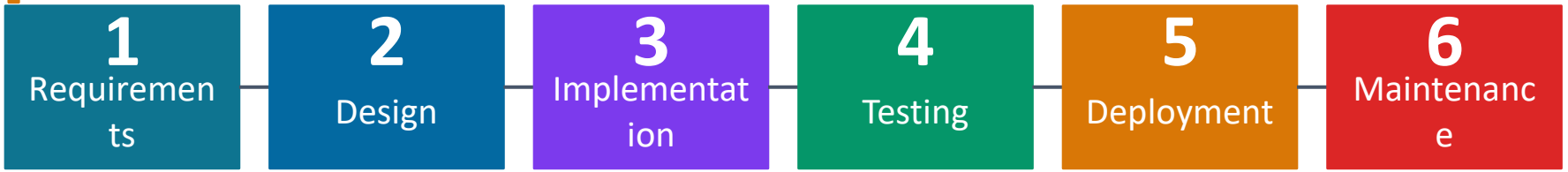
Development Methodology?

- A **software development methodology** is a structured approach that guides how software is planned, developed, and delivered. It defines the **phases**, **practices**, and **tools** used throughout the project.
- Examples: Waterfall, Agile, DevOps

- The development process is often viewed as a **life cycle**, from initial concept to retirement of the software. Using the right methodology helps manage complexity, improve quality, and ensure project success.

The Software Development Life Cycle (SDLC)

A structured approach to building software — not just writing code



Requirements:

Understand what the system must do. Gather user needs and document specifications.

Design:

Define architecture, components, data structures and interfaces.

Implementation:

Write the actual code based on the design specifications.

Testing:

Verify the system works correctly and meets requirements.

Deployment:

Release to users: install, configure, train.

Maintenance:

Fix bugs, add features, and adapt to new environments.

We will study process models (Waterfall, Agile, DevOps) in depth in Section 2.

Who Builds Software?

A software project team — roles and responsibilities

Project Manager

Oversees planning, timelines, budget, and team coordination

Systems Analyst

Gathers requirements and defines system specifications

Software Architect / Designer

Creates system architecture and user interface design

Programmer / Developer

Writes, builds, and integrates the code

QA Engineer / Tester

Tests software to ensure quality, correctness, and functionality

Technical Writer / Support

Manages documentation and assists administrative tasks

Software Project Success & Failure Rates

- 📌 **Successful** = On time, on budget, with full scope
  ⚠️ **Challenged** = Delivered but over budget, late, or missing features
  ❌ **Failed** = Cancelled or never used
- **Latest Reality (2023–2025 range)**

Status	Approx. %
Successful	35–45%
Challenged	40–50%
Failed	10–20%

What Makes a Project Succeed — or Fail?

Signs of Success

- Delivered on agreed time
- Costs remain within budget
- Meets user expectations fully
- Motivated, collaborative team
- Well-documented and maintainable code

Common Failure Causes

- Unclear or constantly changing requirements
- Poor communication between all stakeholders
- Unrealistic timelines or budgets set at start
- Lack of testing discipline — defects found late
- Scope creep — features keep expanding
- No formal engineering process followed

What is Software Quality?

Quality is not accidental — it is engineered in

Good software is not just software that works. It delivers the right thing, reliably, efficiently, and in a form that can be maintained over time.

Product Quality

Characteristics of the delivered software

- Usability — easy and intuitive for users
- Reliability — fails rarely and recovers well
- Efficiency — performs well under normal load
- Maintainability — easy to understand and fix
- Security — protects user data and system

Process Quality

Characteristics of how it was built

- Standards followed consistently
- Code reviews and inspections carried out
- Testing at every stage of development
- Documentation kept up to date
- Version control used throughout

"Think of product quality as how good the cake is, and process quality as how clean and organised the kitchen was. You need both for consistent results."

What Changed Since Early 2000s?

- **Better tools** (e.g., Jira, Git, CI/CD)
- **Agile and DevOps adoption**
- **Improved communication (Slack, Zoom)**
- **Cloud platforms reducing infrastructure risks**

These advances **increased success** and **reduced failure**, but challenges like scope creep, unrealistic expectations, and unclear requirements still persist

1.7

Professional & Ethical Responsibility

The human dimension of software engineering

Professional & Ethical Responsibility

Software engineering is about trust, safety, and public good — not just code

What you build could affect thousands — or millions of people. That responsibility shapes how you must work.

Confidentiality

Respect sensitive user and organisational data, even without a formal NDA

Competence

Do not accept work you are not qualified to do — seek help or be honest

Intellectual Property

Respect copyright, licences, and patents — including open source licences

Computer Misuse

Do not abuse system access, install malware, or misuse employer resources

Public Safety

Safety-critical software demands the highest rigour — lives may depend on it

Honesty

Be transparent about progress, problems, and limitations with your team and clients

“Ethics is doing the right thing — even when no one is watching.”

The ACM/IEEE Code of Ethics

8 principles that define professional conduct for software engineers worldwide

Created jointly by ACM and IEEE-CS — the shared global standard for the profession.

1 Public

Act consistently in the public interest — welfare and safety first

2 Client & Employer

Serve clients and employers, consistent with the public interest

3 Product

Ensure the highest possible professional standards in your work

4 Judgement

Maintain integrity and independence in your professional judgement

5 Management

Promote an ethical approach to managing development and teams

6 Profession

Advance the integrity and reputation of software engineering

7 Colleagues

Be fair, honest, and supportive of your fellow engineers

8 Self

Commit to lifelong learning and ethical professional practice

Full reading: ACM/IEEE-CS Software Engineering Code of Ethics and Professional Practice — assigned as additional reading material

1.8

The Modern Software Landscape

Web, cloud, mobile, and AI — software engineering today

How Modern Software is Delivered

The shift from installed products to internet-based services

Web-Based Software

Runs in a browser — no installation needed. Accessible from any device.
Examples: Google Docs, Gmail, online banking, Canva.

Cloud Computing

Software runs on remote servers managed by providers (AWS, Azure, Google Cloud). You pay for what you use.
Examples: Zoom, Netflix, Microsoft 365.

Mobile Applications

Designed for smartphones and tablets. Handles small screens, variable connectivity, and battery life constraints.
Examples: WhatsApp, banking apps, Google Maps.

Software as a Service (SaaS)

Software is a subscription, not a purchase. Accessed on demand without installing a local copy.
Examples: Spotify, ChatGPT Plus, Adobe Creative Cloud.

"Today, software is a service, not a product. You do not install MS Word — you access it. That is web-based software engineering."

Core Principles of Software Engineering

These apply to all software — regardless of tools, language, or methodology

1

Use a Defined Process

Develop using a structured, managed process. Different systems suit different approaches — no single one-size-fits-all method exists.

2

Ensure Dependability

Every system must be reliable (no unexpected failures), secure (protected from threats), and efficient under realistic load.

3

Start with Clear Requirements

Know exactly what to build before writing code. Requirements engineering prevents wasted effort and misaligned products.

4

Reuse When Possible

Use existing libraries, components, and services where appropriate. Do not rebuild what already exists and works well.

Modern Software is Built for the Web & the Cloud

- Web-Based Systems
  - The **Web is the new platform**: Most apps run via browsers, not local installs.
  - Web-based apps = **accessible anywhere**, **cross-platform**, and **easier to update**.

- Web Services
  - Applications expose **functionality via APIs** over the web (e.g., REST, GraphQL).
  - Enables **integration** with other services (payment, maps, social logins).

Cloud Computing

- Software runs on **remote servers**, not personal machines.
- Examples: Google Docs, Zoom, Canva, Microsoft 365.
- Cloud platforms = AWS, Azure, Google Cloud.

Pay-as-you-go Model (SaaS)

- Users **don't buy software outright** — they **subscribe** (e.g., Netflix, ChatGPT).
- Businesses benefit from **scalability, lower upfront costs, and remote access**.

"Today, software is a *service*, not a product. You don't install Photoshop — you access it. That's web-based software engineering."

Web-based Software Engineering

- Web-based systems are **complex, distributed systems** running over the internet.
- Despite their complexity, they follow the **same software engineering principles** (requirements, design, implementation, testing, and maintenance).

- "Web-based systems demand fast, scalable, and modular design, but they are built on the same core engineering values that apply to all software systems — with more agility, service reuse, and richer user interfaces."

Key Engineering Practices

- **1. Software Reuse**
  Web systems often **reuse components** (libraries, APIs, services) instead of building everything from scratch.
- **2. Incremental & Agile Development**
  Requirements change frequently. So, these systems are built **iteratively** using **Agile methods**.
- **3. Service-Oriented Architecture (SOA)**
  Components are designed as **web services** that can function independently (e.g., login service, payment gateway).
- **4. Rich Interfaces**
  Technologies like **AJAX**, **React**, and **HTML5** allow creation of **dynamic and interactive UIs** that work inside the browser.

AI and Software Engineering Today

A new tool — not a replacement for engineering thinking

AI tools like GitHub Copilot and ChatGPT can generate code. Does that mean software engineering no longer matters?

What AI Tools Do Well

- Generate boilerplate code quickly
- Suggest completions for common patterns
- Explain existing code
- Speed up routine implementation tasks
- Help with documentation and test case drafts

Where Engineering is Still Essential

- Understanding user requirements correctly
- Designing reliable, scalable system architecture
- Making software maintainable and secure
- Testing thoroughly and systematically
- Taking professional responsibility for the product

AI generates code. Software engineering ensures that code is correct, safe, reliable, and maintainable.

Section 1 — What We Have Covered

Check your understanding against the learning outcomes

- **LO1** Define software and software engineering
- **LO2** Classify types of software and their application domains
- **LO3** Explain what makes software development uniquely challenging
- **LO4** Describe the software crisis and how software engineering emerged
- **LO5** Outline SDLC stages and software project team roles
- **LO6** Identify ethical responsibilities of software engineers
- **LO7** Describe how modern software is delivered — web, cloud, mobile, AI

Coming next — Section 2: Software Processes (Waterfall, Agile, DevOps, and process activities)

FAQs

Question	Answer
What is software?	Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.
What are the attributes of good software?	Good software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable.
What is software engineering?	Software engineering is an engineering discipline that is concerned with all aspects of software production.
What are the fundamental software engineering activities?	Software specification, software development, software validation and software evolution.
What is the difference between software engineering and computer science?	Computer science focuses on theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.
What is the difference between software engineering and system engineering?	System engineering is concerned with all aspects of computer-based systems development including hardware, software and process engineering. Software engineering is part of this more general process.

FAQs

Question	Answer
What are the key challenges facing software engineering?	Coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.
What are the costs of software engineering?	Roughly 60% of software costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.
What are the best software engineering techniques and methods?	While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system. For example, games should always be developed using a series of prototypes whereas safety critical control systems require a complete and analyzable specification to be developed. You can't, therefore, say that one method is better than another.
What differences has the web made to software engineering?	The web has led to the availability of software services and the possibility of developing highly distributed service- based systems. Web-based systems development has led to important advances in programming languages and software reuse.

References & Further Reading

Main Textbooks

Software Engineering

Ian Sommerville, 10th edition, Addison-Wesley, 2015.

Chapters 1 and 2 assigned as additional reading for this section

Software Engineering: A Practitioner's Approach

Roger S. Pressman, 9th edition, McGraw-Hill International, 2020.

Chapters 1 and 2 assigned as additional reading for this section

Standards & Professional Codes

- ACM/IEEE-CS Software Engineering Code of Ethics and Professional Practice
- IEEE Std 610.12 — IEEE Standard Glossary of Software Engineering Terminology